

Grado en Ingeniería informática
2025-2026

Trabajo Fin de Grado
Exploiting Retrieval Latency Side-Channels in
RAG Systems.

Sergio Verde Llorente
Tutor/es

José María de Fuentes

Lugar y fecha de presentación prevista



[Incluir en el caso del interés de su publicación en el archivo abierto]

Esta obra se encuentra sujeta a la licencia Creative Commons **Reconocimiento – No Comercial – Sin Obra Derivada**

ÍNDICE

1. INTRODUCCIÓN	4
2. ESTADO DEL ARTE.....	6
2.1. PRIVACIDAD Y SEGURIDAD EN SISTEMAS RAG	6
2.1.1. El Conflicto entre Utilidad y Privacidad en la Recuperación Vectorial	6
2.1.2. Explotación de Restricciones de Generación y Presupuesto de Tokens.....	7
2.2. ATAQUES DE CANAL LATERAL EN MODELOS DE LENGUAJE	8
2.2.1. Fundamentos de la Fuga de Información Temporal en la Inferencia Autoregresiva	9
2.2.2. Evaluación sobre Modelos Comerciales y Estrategias de Mitigación	10
2.3. VULNERABILIDADES EN MECANISMOS DE CACHÉ	11
2.3.1. Arquitectura de Memoria y Jerarquía de Caché en LLMs: KV-Cache y Prompt Caching	11
2.3.2. cache semantica o de prefijos Explotación de la Latencia en Cachés Compartidas: El ataque InputSnatch	12
2.3.3. cache kv Fugas de información en entornos multi-inquilino: El paradigma del NDSS 2025	12
2.4. COMPARATIVA DE TRABAJOS RELACIONADOS Y BRECHA DE INVESTIGACIÓN (RESEARCH GAP)	13
2.4.1. Análisis Multidimensional de las Metodologías de Exfiltración.....	13
2.4.2. Cuadro Comparativo de Trabajos Relacionados	14
2.4.3. Análisis de Lagunas y Puntos Ciegos en la Literatura Actual	15
2.4.4. Identificación y Definición de la Brecha de Investigación (The Research Gap).....	16
3. ANÁLISIS DE REQUISITOS	16
3.1. DEFINICIÓN Y CLASIFICACIÓN DE REQUISITOS	17
3.1.1 Requisitos funcionales	17
3.1.2. Requisitos no funcionales.....	18
3.2. MODELO DE CASOS DE USO.....	19
3.2.1. Actores	19
3.2.2. Diagrama de casos de uso	19
3.2.3. Descripción de alto nivel de los casos de uso.....	20
3.3. MATRIZ DE TRAZABILIDAD	21

1. Introducción

- 1.1. Contexto y Motivación
- 1.2. Definición del Problema
- 1.3. Objetivos del Proyecto (General y Específicos)
- 1.4. Metodología y Planificación

- 1.5. Estructura de la Memoria
2. **Estado del Arte y Conceptos Teóricos**
 - 2.1. Generación Aumentada por Recuperación (RAG)
 - 2.2. Modelos de Lenguaje de Gran Escala (LLMs)
 - 2.3. Bases de Datos Vectoriales y Embeddings
 - 2.4. Mecanismos de Caché en Sistemas de IA
3. **Análisis de Requisitos**
 - 3.1. Descripción de Usuarios y Stakeholders
 - 3.2. Casos de Uso (Diagramas y Descripción)
 - 3.3. Requisitos Funcionales
 - 3.4. Requisitos No Funcionales (Rendimiento, Latencia, Escalabilidad)
 - 3.5. Requisitos de Datos y Seguridad
 - 3.6. Análisis de Tecnologías Candidatas
4. **Diseño del Sistema**
 - 4.1. Arquitectura General del Sistema (Arquitectura de Alto Nivel)
 - 4.2. Diseño de Componentes (Módulo RAG, Orquestador, Capa de Datos)
 - 4.3. Diseño de la Capa de Caché Semántica (Lógica de similitud y umbrales)
 - 4.4. Diseño de la Base de Datos y Modelo de Datos Vectorial
 - 4.5. Selección Final de Tecnologías (Justificación técnica)
 - 4.6. Interfaz de Usuario / API de Exposición
5. **Implementación y Desarrollo**
 - 5.1. Configuración del Entorno de Desarrollo y Despliegue (Docker/RunPod)
 - 5.2. Desarrollo del Pipeline de Ingesta (ETL de documentos)
 - 5.3. Desarrollo del Motor de Inferencia y RAG
 - 5.4. Integración de Redis como Caché Semántica
 - 5.5. Gestión de Errores y Excepciones
6. **Pruebas y Evaluación (Experimentación)**
 - 6.1. Definición del Escenario de Pruebas y Datasets
 - 6.2. Pruebas de Funcionamiento (Caja Negra)
 - 6.3. Evaluación del Rendimiento (Latencia con vs. sin Caché)
 - 6.4. Análisis de Calidad de las Respuestas (Métricas RAG)
 - 6.5. Resultados y Discusión
7. **Aspectos Económicos y Sociales**
 - 7.1. Presupuesto del Proyecto (Costes de HW, SW y personal)
 - 7.2. Análisis de Impacto Social y Ético (Uso de IA y privacidad)
 - 7.3. Impacto Ambiental (Consumo energético de la inferencia)
8. **Conclusiones y Trabajo Futuro**
 - 8.1. Conclusiones y Consecución de Objetivos
 - 8.2. Limitaciones del Sistema
 - 8.3. Líneas de Trabajo Futuras
9. **Bibliografía**
10. **Anexos**
 - 10.1 Manual de Instalación y Usuario
 - 10.2 Código Fuente Relevante / Repositorio
 - 10.3 Capturas de Pantalla y Pruebas Detalladas

1. INTRODUCCIÓN

El proyecto se centra en investigar un posible fallo de seguridad en los sistemas RAG, que son los que usan las empresas para que la IA lea sus propios documentos. Mi enfoque es diferente a lo habitual ya que quiero fijarme en la base de datos donde se guarda la información. El escenario real que planteo es el de una empresa con un chat interno donde se mezcla información pública con archivos confidenciales que no todos deberían saber que existen.

Para poner esto a prueba, voy a realizar un experimento lanzando "preguntas trampa" sobre temas sensibles (como por ejemplo "listado de despidos") y voy a medir con mucha precisión cuánto tarda el sistema en procesar la búsqueda. Mi teoría es que los algoritmos que usan estas bases de datos no tardan siempre lo mismo: creo que cuando encuentran documentos sobre un tema, les cuesta un poco más de trabajo procesarlos que cuando buscan en una zona vacía.

El objetivo final es comprobar si esas pequeñas diferencias de tiempo son reales y medibles. Quiero ver si un atacante podría llegar a deducir que un documento secreto existe simplemente cronometrando la espera, incluso si al final el chatbot responde con un "no tengo permisos para ver esto". Básicamente, busco ver si la velocidad de la búsqueda delata la existencia del archivo.

Tabla de comparación con trabajos científicos.

No he podido encontrar trabajos del tema exacto que quiero tratar, pero he encontrado muchos trabajos que utilizan cosas similares. Por ejemplo, hay varios que utilizan el tiempo a primer token, que es lo que yo quiero medir, pero lo utilizan para medir tiempo de acierto a cache o para clasificar un tema. Otro también hace búsqueda de datos en la Retrieval Database pero lo hace mirando la confianza de respuesta de la IA. Además, no he encontrado trabajos de las revistas que nos ofreciste, pero he seleccionado los de arXiv que me parecían más potentes o que tenían sentido que fueran buenos por las personas que los hacen. La explicación de porque los he seleccionado se encuentra entre paréntesis en la columna de revista. He añadido una descripción de cada paper más detallada después de la tabla ya que no tenía hueco en la misma.

Año	Título	Revista	Componente Atacado	Características	Señal Medida	Link
2025	Is My Data in Your Retrieval Database? Membership Inference Attacks Against Retrieval Augmented Generation	arXiv (Harvard University)	Salida del LLM (Probabilidad)	Usa estadísticas de texto (confianza/entropía) para inferir datos.	log-probabilities seguridad de la IA al responder.	https://arxiv.org/abs/2405.20446

2024	InputSnatch: Stealing Input in LLM Services	arXiv (Tema similar, ya que busca si algo esta guardado, pero con la cache y con prompts en vez de con archivos)	Memoria Caché	Envía sondas para detectar si una respuesta es "instantánea", indicando que otro usuario ya hizo esa consulta antes.	Tiempo de Acierto en Caché	https://arxiv.org/abs/2411.18191
2025	BudgetLeak: Membership Inference on RAG	arXiv (Usa RAG solo que con otro tipo de ataque pero centrándose en el conocimiento)	LLM (Fase de Generación)	Manipula el presupuesto de tokens para ver si el LLM cambia su comportamiento al acceder a un dato privado.	Tokens usados	https://arxiv.org/abs/2511.12043
2025	WhisperLeak: Side-channel attack on LLM	arXiv (Microsoft) (Las empresas han hecho cambios gracias a este trabajo)	Tráfico de Red	Analiza patrones de tiempo y tamaño de paquetes en respuestas token a token cifradas para clasificar el tema.	Patrón de Tráfico	https://arxiv.org/abs/2511.03675
2025	Mi proyecto	N/A	Base de Datos Vectorial	Explota la carga computacional variable de los algoritmos de búsqueda dependiendo de la densidad de los datos.	Tiempo de Búsqueda	N/A

2. ESTADO DEL ARTE

En este capítulo se realiza una revisión crítica de la literatura científica más reciente relacionada con la seguridad en arquitecturas de inteligencia artificial generativa. Concretamente, se analizarán los avances y vulnerabilidades descubiertas en los sistemas de Generación Aumentada por Recuperación (RAG), los ataques fundamentados en canales laterales (*side-channel attacks*) sobre Modelos de Lenguaje de Gran Escala (LLMs), y las brechas de privacidad emergentes en los mecanismos de memoria caché. Este análisis permitirá contextualizar el presente Trabajo de Fin de Grado y justificar la relevancia de la brecha de investigación abordada.

2.1. PRIVACIDAD Y SEGURIDAD EN SISTEMAS RAG

Los LLMs han revolucionado el procesamiento del lenguaje natural. Sin embargo, se ha visto con el paso del tiempo que tienen tendencia a generar información falsa por las alucinaciones. Por esto, muchas empresas están implementando en sus sistemas de *chatbots* de IA las arquitecturas RAG. Estas ayudan a mejorar ampliamente la precisión de los LLMs al utilizar partes de documentos de una base vectorial como contexto para el LLM. El principal problema es que, como podemos ver en varias investigaciones recientes, introducen muchos problemas de seguridad y privacidad en los sistemas.

2.1.1. EL CONFLICTO ENTRE UTILIDAD Y PRIVACIDAD EN LA RECUPERACIÓN VECTORIAL

El modelo de arquitectura RAG aparece como una solución a las limitaciones de los modelos de lenguaje pre-entrenados. Algunos de los problemas que estos sistemas solucionan son las alucinaciones y falta de veracidad, conocimiento obsoleto o desactualizado y la falta de precisión en dominios de datos privados. Al integrar un sistema de recuperación que extrae fragmentos de documentos desde una base de datos vectorial para contextualizar el *prompt* del modelo, se consigue una precisión muy superior. Sin embargo, este proceso crea una nueva ventana de ataque al unir directamente el contenido de la base de datos al modelo de lenguaje.

La principal amenaza sobre estas arquitecturas actualmente se materializa a través de los Ataques de Inferencia de Membresía (*Membership Inference Attacks* o MIA). Estos ataques tienen como objetivo descubrir si un documento específico está indexado en la base de datos de recuperación de una organización. Cualquier ataque que afirme que un documento confidencial se encuentra o no en una base de datos ya genera una violación grave de la privacidad. Justo esto es lo que se estudia en el paper *Is My Data in Your Retrieval Database?* [1]. Este paper ha demostrado cómo las probabilidades de predicción de los tokens (*log-probabilities*) emitidas por el LLM pueden revelar la presencia de archivos corporativos en una base de datos. Las *log-probabilities* en un LLM son las probabilidades que un modelo asigna a cada token posible al generar la siguiente unidad de texto. Técnicamente, representan la

confianza del modelo en su predicción, indicando mayor confianza cuanto más cercana al 0 sea.

Esta investigación explica que el problema no está en el software del modelo, sino en el funcionamiento de la arquitectura interna del mismo. Cuando el sistema usa un documento real en su contexto, el LLM muestra mayor "facilidad" para predecir los siguientes tokens, lo que se traduce en unas *log-probabilities* más cercanas a 0. Para evaluar esto, los autores utilizaron dos escenarios distintos:

1. Ataques de Caja Negra: En este escenario, el atacante solo tiene acceso al texto final generado por el chat. A pesar del poco conocimiento sobre el sistema, el estudio revela que, mediante el análisis de la salida y su abundancia de detalles, el atacante puede entrenar clasificadores que detecten la aparición de un documento. Sin embargo, al aplicar este método encontraron el obstáculo de las alucinaciones, en las que el modelo generaba respuestas con seguridad alta sobre datos desconocidos o sin contexto, lo que elevaba la tasa de falsos positivos. Aun así, en pruebas con modelos como Mistral-7B, se alcanzaron valores de AUC-ROC cercanos a 0.8, lo que demuestra que, incluso sin ver los parámetros internos, el sistema filtra información.
2. Ataques de Caja Gris: Este escenario también es realista, ya que hay modelos que devuelven metadatos como las *log-probabilities* al usarlos a través de sus APIs. En este caso, el atacante tenía acceso a las *log-probabilities* y podía medir con precisión cuánta confianza tenía el modelo en cada palabra. En el paper [1] se diseñaron unas reglas de decisión con esta métrica en la que, si la probabilidad acumulada superaba cierto umbral, el documento se consideraría presente en la base de datos. Los resultados aumentaron considerablemente frente al anterior escenario con una media de 0.9 en AUC-ROC, lo que significa que el ataque es casi infalible.

Por todo esto, los autores concluyen que existe una correlación directa entre la presencia de un documento en la base de datos de recuperación y una disminución drástica en la entropía de la respuesta del modelo. Para aportar fuerza a sus conclusiones, los autores evaluaron el ataque sobre conjuntos de datos de alta sensibilidad. Se usó el dataset *Enron* (correos electrónicos corporativos reales) y *HealthCareMagic* (consultas médicas), que permitió observar que el ataque es especialmente efectivo en textos con terminología técnica. Esto sugiere que cuanto más "único" es el documento que una empresa intenta proteger en su RAG, más fácil es para un atacante detectar su presencia mediante este tipo de canales laterales.

2.1.2. EXPLOTACIÓN DE RESTRICCIONES DE GENERACIÓN Y PRESUPUESTO DE TOKENS

Además del análisis de confianza de una salida, los sistemas de LLMs tienen muchas más métricas que se pueden estudiar, que también suponen un vector de ataque. Una de ellas es la limitación por tokens de los modelos. En entornos reales y plataformas comerciales, la generación de texto está estrictamente limitada por cuotas de tokens para garantizar la eficiencia y, sobre todo, el control de costes. Esta restricción de diseño es la que se usa como base del ataque presentado en el estudio *BudgetLeak: Membership Inference on RAG* [2].

La hipótesis de este trabajo trata sobre la diferencia de comportamiento de un Modelo de Lenguaje de Gran Escala cuando agota la limitación de tokens en la generación de una respuesta frente a cuando la respuesta es menor a la limitación de tokens. Al recibir una consulta sobre un tema específico, si en la arquitectura RAG se encuentra el documento pertinente y lo inyecta en el contexto, el LLM intentará formular una respuesta más larga y completa sobre dichos datos. Sin embargo, si el atacante puede modificar la limitación de tokens reduciéndola a unos pocos tokens, el modelo sufrirá una parada forzada en la generación o usará patrones semánticamente inusuales. Por el contrario, si el documento no se encuentra en la base de datos, el sistema generará una respuesta negando el conocimiento de los datos pedidos, como: "No dispongo de información suficiente", las cuales no suelen consumir los tokens limitantes por completo.

Para probar esta hipótesis planteada, los autores del paper [2] diseñaron dos escenarios de amenaza distintos:

1. **Conocimiento Parcial:** En este escenario, se asume que el atacante posee información sobre la configuración general del RAG objetivo, como la estrategia de fragmentación. Utilizando arquitecturas iguales a la atacada, el atacante entrena un modelo clasificador capaz de identificar los patrones generados por una parada repentina en el texto de salida. Este clasificador aprende a discernir con gran agudeza los defectos que delatan la presencia de información inyectada.
2. **Conocimiento Cero:** Representa el entorno más realista, ya que el ataque se produce sin ninguna noción sobre la infraestructura interna del sistema objetivo. En este caso, el ataque prescinde del entrenamiento supervisado. En su lugar, emplea algoritmos de aprendizaje no supervisado basados en *clustering* aplicados sobre un dataset de respuestas obtenidas tras enviar consultas de manera sistemática. Las métricas muestran que las respuestas contextualizadas por la base de datos vectorial tienden a agruparse en clústeres radicalmente distintos a las respuestas "vacías".

Al realizar este ataque, los investigadores concluyeron que su hipótesis era cierta al observar resultados sorprendentemente altos. Se recogieron valores del 80% al 91% en precisión haciendo las pruebas con bancos de preguntas estandarizados como Natural Questions y MS-MARCO y utilizando datasets de alta privacidad como HealthCareMagic-100k.

2.2. ATAQUES DE CANAL LATERAL EN MODELOS DE LENGUAJE

Como se ha mencionado en la sección anterior, las estrategias para comprometer la privacidad de un sistema basado en LLM dependen de la interacción directa con los parámetros del modelo, analizando sus respuestas textuales o sus restricciones de tokens. Sin embargo, la implementación de Modelos de Lenguaje de Gran Escala introduce una superficie de explotación que ocurre incluso usando las defensas criptográficas convencionales: los ataques de canal lateral (*side-channel attacks*). Estos ataques no se basan en la explotación de vulnerabilidades en el código o en la ruptura de algoritmos de cifrado, sino en la observación de variables físicas que proporcionan información sobre el estado interno del sistema y los datos procesados. Entre ellos encontramos algunos como el consumo energético o las latencias y el tráfico de red.

2.2.1. FUNDAMENTOS DE LA FUGA DE INFORMACIÓN TEMPORAL EN LA INFERENCIA AUTOREGRESIVA

Para comprender la viabilidad de un ataque de canal lateral sobre un modelo de lenguaje, es necesario analizar la arquitectura de la fase de inferencia. La mayoría de los LLMs modernos funcionan de forma autorregresiva, esto quiere decir que el modelo genera el texto secuencialmente, palabra por palabra, contextualizando cada nueva predicción con la secuencia previamente generada. Esta generación se divide en dos fases:

1. Fase de prellenado: En esta etapa, el sistema procesa el *prompt* inicial del usuario de manera paralela. Para ello se cargan los pesos del modelo en la GPU y se calculan las representaciones matemáticas de cada palabra. El tiempo de esta fase es proporcional a la longitud de la consulta.
2. Fase de decodificación: Aquí es donde reside la principal vulnerabilidad de canal lateral. El modelo genera el texto token a token. Cada nuevo token requiere una pasada hacia delante por el sistema. El sistema accede constantemente a la VRAM para recuperar los pesos y el historial de la conversación, conocido como *KV-Cache*.

Es en la fase de decodificación donde la latencia inter-token (el tiempo transcurrido entre la emisión de una palabra y la siguiente) se convierte en una señal no constante, ya que tiene variaciones sutiles que dependen de la arquitectura. Cosas como la dificultad semántica de la palabra a predecir o la complejidad del razonamiento requerido hacen que estas variaciones ocurran en la generación. Cuando estos modelos se exponen a través de internet mediante conexiones cifradas (TLS/HTTPS), el texto plano queda protegido, pero los tiempos de llegada de los paquetes de red permiten reconstruir patrones semánticos complejos sin necesidad de descifrar el contenido.

La investigación más importante en este campo la vemos en el trabajo *WhisperLeak: Side-channel attack on LLM* [3], el cual se centra en el análisis de las señales de red y no en el análisis del texto generado. En este estudio se demuestra que un atacante, situado en un nodo intermedio de la red, solo capaz de escuchar el tráfico de red entre el servidor y el usuario, es capaz de clasificar el contenido del que se habla en una conversación con una precisión muy alta. El ataque se centra en capturar los paquetes de red, quitando el tráfico de control para solo usar los paquetes de datos que transportan los tokens. Además, el ataque se centra exclusivamente en dos variables: el tamaño de los paquetes de red y el tiempo entre las llegadas de las respuestas de cada token o tiempo de inter-llegada.

- **Tamaños de paquete:** Aunque los tokens se empaquetan en tramas cifradas de tamaño variable, la arquitectura del LLM y la configuración del servidor fuerzan agrupaciones de tokens que generan firmas de tamaño predecibles.
- **Patrones de inter-llegada:** Representa el tiempo con el que el motor de inferencia actúa. En este estudio se observó que temas con terminología más concreta y específica, como la medicina, provocan una cadencia de generación distinta a la de una conversación casual. Por eso se utilizó como la métrica con mayor peso en el clasificador.

El entorno usado en este estudio fue orientado a detectar actividades de alta privacidad. En este caso se eligió el tema de lavado de dinero, ya que es un tema de valor muy alto para agencias

gubernamentales y entidades financieras. Para hacer esto se usó el dataset *Quora Questions Pair*, que es una colección de datos diseñada para entrenar modelos de aprendizaje automático en la detección de preguntas duplicadas, que contiene más de 11.000 preguntas sobre diversos temas y se le inyectaron 100 consultas diseñadas para imitar escenarios de lavado de dinero.

Se utilizaron arquitecturas de redes neuronales convolucionales para realizar la clasificación. El clasificador fue capaz de distinguir si las latencias observadas en una traza de red de 30 segundos correspondían a un tema relacionado con el lavado de dinero. Los resultados obtenidos demostraron que, incluso ante la presencia de variaciones normales en el tráfico de internet, la señal emitida por el LLM era suficientemente robusta como para que el ataque mantuviese una tasa de acierto cercana al 95%.

Este hallazgo es muy importante, ya que demuestra que el aislamiento de los modelos no impide que la infraestructura filtre información. El procesamiento de cada consulta es único, pero predecible, dejando una huella que es consistente en diferentes plataformas, tanto de código abierto como privadas.

2.2.2. EVALUACIÓN SOBRE MODELOS COMERCIALES Y ESTRATEGIAS DE MITIGACIÓN

Los ataques de canal lateral no son uniformes para todos los sistemas de LLMs ya que dependen plenamente de la optimización del software y de la infraestructura. En *WhisperLeak* [3], se hizo un análisis con una gran batería de pruebas a diferentes modelos comerciales como Gemini, Grok o modelos de OpenAI. En todos ellos se reportaron valores muy altos de precisión y recall. Además, se analizó el ataque cuando se utilizaba por separado el tamaño de los paquetes y la latencia, y pudieron observar que también se conseguía comprometer la privacidad del usuario, aunque no tanto como con los dos datos juntos.

La importancia de estos hallazgos provocó la necesidad de comunicar a las diferentes empresas los problemas con sus modelos, llevando a los proveedores de estos a implementar parches de seguridad de urgencia. Las medidas utilizadas en estos casos se clasifican dentro de las técnicas de ofuscación de tráfico. En este caso se implementó una medida para cada dato utilizado:

1. Solución para tamaño de paquetes: La técnica utilizada fue el empaquetado de paquetes usando *padding*. Esta técnica consiste en modificar la capa de transporte para que todos los paquetes de red tengan un tamaño uniforme, independientemente del número de tokens que transporten. Si bien esto elimina la "firma de tamaño", no soluciona el problema de la cadencia temporal.
2. Solución para tiempo inter-token: Se utilizó la inyección de retrasos o *jitter*. Esto consiste en introducir retrasos aleatorios artificiales en la fase de decodificación. El *jitter* debe ser lo suficientemente grande como para evadir la clasificación del atacante, pero lo suficientemente pequeño como para que no suponga un problema en la UX. Sin embargo, se ha demostrado que, para que el ruido sea efectivo, la latencia del sistema tendría que incrementarse entre un 200% y un 300%.

El principal problema de estas soluciones es que perjudican directamente al usuario final. Añadir atrasos aleatorios o incrementar el tamaño de los paquetes empeora drásticamente la latencia del usuario final y aumenta el consumo del proveedor. Para una empresa que utiliza RAG para analizar miles de documentos, un incremento de la latencia es inaceptable. El tiempo

de respuesta es una de las métricas más importantes en el mercado de los LLMs. Por tanto, la mayoría de las mitigaciones actuales son insuficientemente buenas y sugieren que el canal lateral temporal es una vulnerabilidad crítica de los sistemas autorregresivos que no puede eliminarse de forma competente sin un cambio de la arquitectura interna.

2.3. VULNERABILIDADES EN MECANISMOS DE CACHÉ

El rendimiento de los modelos de Inteligencia Artificial ha hecho que sea necesaria una gestión avanzada de la memoria. Como en los LLMs la latencia es el principal problema en la experiencia de usuario, las empresas se han centrado en reducirla lo máximo posible. Para esto se han empezado a implementar en las arquitecturas los sistemas de cachés para almacenar algunos resultados precomputados y así evitar la repetición del proceso de inferencia en preguntas similares. El principal problema de esta implementación es la introducción de una nueva vulnerabilidad en los sistemas, ya que la diferencia de tiempo entre acceder solo a la caché para recuperar una respuesta y realizar el proceso completo de inferencia para generar una nueva es muy grande, además de determinista. En esta sección se analizará la infraestructura implementada para reducir la latencia, así como cómo esta puede comprometer los datos privados de conversación de los usuarios.

2.3.1. ARQUITECTURA DE MEMORIA Y JERARQUÍA DE CACHÉ EN LLMS: KV-CACHE Y PROMPT CACHING

Para entender cómo un sistema de caché puede generar una vulnerabilidad, es necesario entender más a fondo los procesos de una arquitectura de este estilo. En este caso nos centraremos en el coste de los procesos, ya que las cachés se aplican principalmente para reducir los más costosos. En este caso, el proceso más costoso dentro de la inferencia es el cálculo de relevancia, que ocurre tanto en la fase de prellenado (solo una vez para todo el prompt) como en la fase de decodificación (cada vez que se genera una nueva palabra). Este proceso requiere la computación de grandes matrices de Claves y Valores para cada token de la secuencia. Es por esto que se utilizan mecanismos de caché para poder guardar el contexto computado y así no tener que hacer de nuevo los cálculos completos. Los mecanismos que intervienen en este proceso son:

1. KV-Cache (Caché de Claves y Valores): Esta caché es generada en el proceso de prellenado y guarda las matrices calculadas de Claves y Valores del prompt entregado al sistema. Esta caché utiliza la VRAM de la GPU y se usa en el proceso de la decodificación. De este modo, el sistema, al generar un nuevo token, solo tendrá que calcular la relevancia para el nuevo token y concatenarla a la existente en la caché. Esto reduce considerablemente la necesidad de cómputo y, por lo tanto, el coste del proceso.
2. Prompt Caching y Prefix Caching: Este mecanismo de caché funciona a un nivel mayor, ya que se basa en el funcionamiento de cualquier sistema de caché. En este caso, este se implementa para que, si dos usuarios hacen consultas sobre el mismo contexto, por ejemplo, sobre el mismo documento en un RAG, se guarde el estado del prefijo, o sea, de la primera parte del contexto. Por lo que el primer usuario sí que tendrá un coste temporal mayor, pero el resto que pregunten sobre el mismo documento no.

Estos dos mecanismos hacen que el funcionamiento de los sistemas con LLMs sea mucho mejor al reducir drásticamente su latencia, pero son justo los mecanismos que hacen que el

ataque sea posible. Ya que, si una consulta se procesa utilizando la caché, su Tiempo hasta el Primer Token (*Time To First Token*) será mucho menor. Por lo que permite a los atacantes inferir si algo ha sido procesado con anterioridad o no, rompiendo de nuevo la privacidad del sistema.

2.3.2. CACHE SEMANTICA O DE PREFIJOS EXPLOTACIÓN DE LA LATENCIA EN CACHÉS COMPARTIDAS: EL ATAQUE INPUTSNATCH

La investigación de esta amenaza se realiza en el trabajo *InputSnatch: Stealing Input in LLM Services* [4]. En este trabajo se habla de cómo las cachés compartidas en sistemas de LLMs pueden usarse para reconstruir los *prompts* o temas de consulta de otros usuarios. En este caso en concreto se estudian las cachés de sistemas *multi-tenant* donde distintos usuarios acceden al mismo motor de inferencia.

El ataque propuesto en el paper [4] se basa en la medición del tiempo de respuesta utilizando consultas diseñadas específicamente para que contengan fragmentos de información sensible. Si el sistema devuelve la respuesta con una latencia extremadamente baja, el atacante puede afirmar que ha ocurrido un acierto en caché, lo que implica que otro usuario ha introducido previamente ese mismo fragmento o uno muy similar. El estudio se centra en observar estas mediciones y demostrar cómo la diferencia entre el acierto en caché (*Cache Hit*) y el fallo en caché (*Cache Miss*) es suficientemente amplia como para medirlas incluso a través del ruido de internet. Este ataque fue validado tanto con modelos *open source*, utilizando modelos de la familia LLaMa, como con modelos comerciales de OpenAI.

Para aportar robustez al trabajo, se usó un dataset de interacciones entre médicos y pacientes llamado *Chatdoctor* con más de 110.000 interacciones. Los investigadores demostraron que, mediante el envío de consultas específicas relacionadas con síntomas o enfermedades raras, un atacante puede determinar si un paciente en concreto ha consultado algo sobre una patología concreta observando únicamente la velocidad de respuesta del modelo.

Es relevante mencionar que una de las conclusiones que se saca en el paper [4] es que en la obtención del tema, ya sea más genérico o específico como un diagnóstico en concreto, el ataque es mucho más efectivo que en la reconstrucción de un prompt palabra por palabra. Esto ocurre, ya que las cachés utilizadas hoy en día en este tipo de sistemas suelen ser cachés semánticas. Las cachés semánticas son cachés que están diseñadas para reconocer redundancias a nivel semántico, lo que quiere decir que trabajan con un umbral de similitud y, si ese umbral es superado, la caché genera un hit aunque lo que se compare no sea exactamente igual.

2.3.3. CACHE KV FUGAS DE INFORMACIÓN EN ENTORNOS MULTI-INQUILINO: EL PARADIGMA DEL NDSS 2025

Otro estudio reciente muy relevante es el titulado *"I Know What You Asked: Prompt Leakage via KV-Cache Sharing in Multi-Tenant LLM Serving"* [5]. En este se busca algo similar al anterior, pero en vez de atacar a la caché de prefijos, se utiliza la KV-Cache. De nuevo, lo que se usa es una arquitectura *multi-tenant*, ya que son las más utilizadas. Estas se usan para ahorrar recursos y hacer que varios usuarios compartan el mismo hardware y así poder maximizar el uso de la VRAM de cada GPU.

El ataque que se estudia en este paper [5] se basa en que el atacante envíe múltiples candidatos de tokens y mida el *Time To First Token*. Si detecta una latencia muy baja, confirma un acierto

en caché, lo que le indica que ese token específico es el que la víctima ha utilizado en ese momento. Al repetir este proceso, el atacante consigue reconstruir el *prompt* privado palabra por palabra. En este caso, el ataque se lleva a cabo en distintos escenarios, pero incluso en el peor de todos, en el que el atacante no conoce nada del usuario ni de lo que está diciendo, se consigue una precisión del 95% en la reconstrucción del *prompt* del usuario. Para aplicar robustez en este ataque, solo se asume que el atacante sabe cosas públicas del sistema, como por ejemplo la forma de tokenizar que tiene el modelo.

La integración de bases de datos vectoriales con mecanismos de caché es lo mejor para la eficiencia en las arquitecturas RAG actuales. Sin embargo, tras analizar los trabajos [4, 5], es bastante claro que esta aceleración del mecanismo solo en situaciones específicas es a su vez un generador de vectores de ataque a la privacidad del usuario.

2.4. COMPARATIVA DE TRABAJOS RELACIONADOS Y BRECHA DE INVESTIGACIÓN (RESEARCH GAP)

Después de haber explicado las vulnerabilidades de los sistemas RAG y todos los ataques principales que las envuelven. En este apartado estudiaremos las limitaciones existentes y explicaremos con precisión el punto en donde este trabajo es relevante.

2.4.1. ANÁLISIS MULTIDIMENSIONAL DE LAS METODOLOGÍAS DE EXFILTRACIÓN

La revisión del estado del arte permite agrupar las amenazas contra la privacidad en LLMs en tres dimensiones metodológicas claramente diferenciadas:

1. Ataques de inferencia basados en análisis textual: Los trabajos centrados en ataques de inferencia de membresía (MIA) [1], [2] se basan en interactuar con el modelo y analizar el texto que este devuelve. Estos ataques estudian cómo responde la IA o analizan los valores de confianza matemática que el modelo asigna a cada palabra generada. Son métodos altamente efectivos para comprobar si un párrafo específico ha sido proporcionado al modelo como contexto. Sin embargo, su principal carencia es la escalabilidad. Para que funcionen bien, el auditor necesita hacerle muchísimas preguntas al modelo (interacciones largas) o tener acceso a los valores matemáticos internos que los modelos comerciales suelen ocultar por seguridad. Por tanto, aunque son excelentes para validar un documento suelto, resultan demasiado lentos e inviables para auditar una base de datos corporativa masiva.
2. Ataques de canal lateral en tráfico de red: Investigaciones como *WhisperLeak* [3] cambian el enfoque, ya que en lugar de mirar qué dice el texto, analizan cómo viaja por internet. Su gran fortaleza es que son ataques pasivos. El atacante no necesita enviarle preguntas a la IA, solo observar el tráfico de red cifrado, midiendo el tamaño de los paquetes de datos y el tiempo entre ellos. Estos ataques son muy eficaces para interceptar conversaciones y deducir el tema general del que se está hablando. El problema es que las conexiones a internet tienen fluctuaciones y retrasos, lo que las hace geniales para adivinar contextos amplios, pero no para tener precisión sobre si un documento confidencial se encuentra indexado en el sistema RAG de una empresa o no.

- Ataques centrados en la arquitectura: Los estudios más recientes sobre el uso compartido de cachés [4], [5] se centran en medir las diferencias exactas en los tiempos de respuesta del servidor. Como han demostrado, son herramientas tremendamente precisas que incluso permiten reconstruir palabra por palabra lo que otro usuario está escribiendo en ese instante. Su limitación radica en su objetivo y requisitos. Estos trabajos están orientados a espiar a otros usuarios (robar *prompts*) y suelen requerir que el atacante y la víctima compartan la misma tarjeta gráfica en el servidor.

2.4.2. CUADRO COMPARATIVO DE TRABAJOS RELACIONADOS

A continuación, se presenta la Tabla I, que sintetiza los parámetros técnicos de las investigaciones más relevantes analizadas hasta la fecha. Esta tabla permite visualizar las diferencias en cuanto a objetivos, vectores de ataque y requisitos de hardware, sirviendo como base para la identificación de la brecha de investigación.

Referencia	Vector de Ataque	Objetivo Primario	Métrica de Fuga	Dependencia de Hardware
[1] MIA in RAG	Interfaz Lógica	Inferencia de Membresía	<i>Log-probabilities</i>	Baja (Caja Negra/Gris)
[2] BudgetLeak	Restricciones API	Inferencia de Membresía	Presupuesto de Tokens	Baja (API)
[3] WhisperLeak	Canal Lateral (Red)	Clasificación de Temas	Tiempo y Tamaño	Media (Sniffing)
[4] InputSnatch	Canal Lateral (Caché)	Robo de Inputs/Prompts	Latencia de Respuesta	Alta (Multi-tenant)
[5] NDSS 2025	Arquitectura	Reconstrucción de Prompt	KV-Cache / Latencia	Muy Alta (GPU/VRAM)
Propuesta TFG	Canal Lateral (Tiempo)	Detección de Documentos Indexados	Latencia Semántica	Media (Redis/RAG)

Este análisis comparativo revela que existe una progresión hacia el uso de métricas temporales de baja latencia debido a su alta fiabilidad. Sin embargo, se observa una desconexión entre los ataques de microarquitectura (extremadamente complejos y dependientes de compartir una GPU físicamente) y los ataques de red (demasiado ruidosos). La oportunidad de investigación se encuentra en un punto intermedio: el uso de la **latencia de la caché semántica** en servidores

RAG, un vector que combina la precisión de los ataques de caché con la facilidad de acceso de los ataques de red.

2.4.3. ANÁLISIS DE LAGUNAS Y PUNTOS CIEGOS EN LA LITERATURA ACTUAL

A pesar de los avances significativos en la identificación de vulnerabilidades dentro del ecosistema de los chatbots, el análisis del estado del arte revela que todavía hay muchas partes de estos sistemas inexploradas que podrían generar nuevos problemas de seguridad y privacidad. En este trabajo se han aprovechado estos puntos no explorados para diseñar y ejecutar un nuevo vector de ataque.

La inmensa mayoría de los ataques enfocados a la arquitectura (como los expuestos en el NDSS 2025 [5] o InputSnatch [4]) dirigen sus esfuerzos exclusivamente hacia la memoria del Modelo de Lenguaje, en concreto a la KV-Cache o al prompt caching. Sin embargo, se ha dado por sentado que el corazón de la arquitectura RAG, el motor de recuperación, es seguro y no se puede usar para realizar estos ataques. Las bases de datos vectoriales modernas (como ChromaDB) utilizan algoritmos de búsqueda aproximada por vecinos más cercanos (ANN), siendo HNSW (Hierarchical Navigable Small World) el estándar. Es aquí justo donde encontramos la principal laguna crítica, ya que hay una notable falta de pruebas sobre este algoritmo.

HNSW optimiza las búsquedas navegando a través de grafos multicapa. El tiempo de recorrido computacional varía de forma determinista dependiendo de si el vector consultado está rodeado de muchos otros o se encuentra en un espacio vacío del RAG. Ya que el algoritmo realiza una búsqueda más larga cuando el espacio de su alrededor se encuentra mayoritariamente vacío, al necesitar encontrar una respuesta más precisa entre vecinos más lejanos. Sin embargo, cuando hay muchos vectores alrededor del buscado, el algoritmo encuentra con rapidez la mejor respuesta. En este TFG, se ha aprovechado precisamente el bajo estudio sobre este algoritmo para convertir sus tiempos de respuesta en un vector de ataque para la inferencia de contenido.

Además, para añadir robustez al trabajo, se ha estudiado si el ataque también se presenta en un sistema con todos los componentes aportando ruido. En los entornos de producción real, un sistema RAG es un pipeline secuencial orquestado por frameworks como LangChain o LlamaIndex (involucrando modelos de embedding, bases de datos y el LLM final). Todas estas partes del sistema aportan ruido al tener su propio tiempo de computación. En este caso se ha estudiado si la diferencia de tiempo de recuperación entre una query cuyo vector cae en una zona poblada del RAG y otra cuyo vector cae en una más vacía es suficientemente grande como para medirla con todo el ruido generado por los distintos componentes del sistema.

Finalmente, la investigación actual se centra en comprometer la privacidad del usuario. Los ataques diseñados tienen como objetivo principal inferir o reconstruir las preguntas que otros usuarios envían al sistema (fuga de inputs o colisiones multi-inquilino). Lo que los estudios han pasado por alto es qué pasaría si, en lugar de utilizar la latencia para "espiar" canales dinámicos (chats activos), nos centramos en explotar las fluctuaciones de tiempo para estudiar componentes estáticos como la base de conocimiento corporativa. Al aprovechar esta falta de atención sobre el corpus, el ataque propuesto logra confirmar la existencia de documentos confidenciales indexados.

2.4.4. IDENTIFICACIÓN Y DEFINICIÓN DE LA BRECHA DE INVESTIGACIÓN (THE RESEARCH GAP)

El conjunto de estas lagunas permite identificar una brecha de investigación clara y de alta relevancia, que es la ausencia del estudio de un método para auditoría de membresía de documentos en sistemas RAG mediante el análisis de latencia del algoritmo de búsqueda vectorial (como HNSW). La investigación propuesta para explotar esta brecha se define con los siguientes elementos:

1. El vector: La latencia generada específicamente por el motor de búsqueda de la base de datos vectorial al procesar la similitud de consultas contra su índice.
2. El objetivo: Descubrir y confirmar la existencia de archivos específicos dentro del índice vectorial de una arquitectura RAG.
3. El contexto: Un entorno de auditoría corporativa realista (que incluye capas de optimización estándar como cachés semánticas) donde se busca validar si el sistema delata la existencia de documentos que no deberían estar expuestos.

Esta investigación, a diferencia de otros ataques, se centra en una vulnerabilidad algorítmica que es la propia eficiencia de la indexación y recuperación vectorial, midiendo el comportamiento del *retriever* y definiendo qué diferencia de tiempo constituye una prueba de la presencia de un documento en el RAG. Además, este trabajo se justifica gracias a su gran innovación en varias pilares:

- Viabilidad y realismo: A diferencia de los ataques presentados en el NDSS 2025[5], esta metodología no requiere acceso físico o lógico a la GPU. Se basa en una interacción de red estándar con el servicio RAG, lo que la hace aplicable a prácticamente cualquier despliegue en la nube (AWS, Azure, Google Cloud).
- Enfoque en la privacidad de los datos: Al orientar el análisis hacia la base de datos vectorial, este trabajo ataca la principal preocupación de las empresas que utilizan sus propios chatbots, el miedo a que datos corporativos confidenciales se filtren por algún problema de seguridad.
- Formalización de la auditoría: No solo se pretende demostrar una vulnerabilidad, sino realizar una herramienta que permita a las empresas auditar sus propios sistemas RAG. También se propondrán algunas soluciones para mitigar la brecha de seguridad.

3. ANÁLISIS DE REQUISITOS

El análisis de requisitos es uno de los pilares fundamentales de cualquier proyecto de software. Este trabajo no solo se centra en la investigación del side-channel attack, sino también en el desarrollo de un producto de software funcional que ayude a la realización del mismo en distintas partes del mundo.

El objetivo de esta sección del proyecto es poder identificar y documentar de manera precisa las necesidades del sistema para que pueda cumplir todos los requisitos de la investigación planteada. Este análisis nos ayudará a transformar las ideas y objetivos del proyecto en especificaciones técnicas concretas para asegurar que las fases de diseño y desarrollo sean exitosas en un futuro.

Para poder asegurar la calidad del proyecto, se ha seguido una metodología basada en estándares de ingeniería del software utilizados en entornos profesionales. Por lo tanto, cada

requisito ha sido evaluado y revisado para asegurar que el resultado final sea un producto de software funcional, claro y útil para la comunidad de investigación de ciberseguridad e inteligencia artificial. Además, todos los requisitos también se han planteado buscando la escalabilidad y robustez del producto.

3.1. DEFINICIÓN Y CLASIFICACIÓN DE REQUISITOS

La identificación de los requisitos se ha realizado siguiendo la nomenclatura **UG-*nnn***, donde la "U" indica un requisito del usuario y la "G" se refiere a un requisito general del sistema. A continuación, se presenta la tabla detallada con los requisitos funcionales y no funcionales que regirán el desarrollo del proyecto.

3.1.1 REQUISITOS FUNCIONALES

Identificador: UG-001	
Nombre: Introducción de términos de consulta	
Prioridad: Alta	Fuente: Cliente
Necesidad: Media	
Claridad: Alta	Verificabilidad: Alta
Estabilidad: Alta	
Descripción:	El usuario dispondrá de un campo de texto en la interfaz gráfica donde podrá introducir el término, palabra clave o frase que desea investigar o auditar contra la base de datos.
Identificador: UG-002	
Nombre: Visualización del estado del sistema	
Prioridad: Media	Fuente: Cliente
Necesidad: Media	
Claridad: Alta	Verificabilidad: Alta
Estabilidad: Alta	
Descripción:	La interfaz mostrará al usuario la información del entorno sobre el que se va a hacer la prueba (ej. si el sistema está conectado, el tipo de base de datos vectorial que hay por debajo y si la caché está activa).
Identificador: UG-003	
Nombre: Cálculo automático de iteraciones necesarias	
Prioridad: Alta	Fuente: Analista
Necesidad: Alta	
Claridad: Alta	Verificabilidad: Alta
Estabilidad: Media	
Descripción:	Basándose en los datos estadísticos previos del sistema, el software calculará y aplicará automáticamente el número de intentos necesarios para obtener una muestra estadísticamente representativa.
Identificador: UG-004	
Nombre: Ejecución automatizada de la auditoría	
Prioridad: Alta	Fuente: Cliente
Necesidad: Alta	
Claridad: Alta	Verificabilidad: Alta

Estabilidad: Alta

Descripción: Tras introducir el término, el usuario podrá lanzar la prueba mediante un botón. El sistema ejecutará automáticamente las consultas RAG repetidas en segundo plano, midiendo los tiempos de latencia de cada iteración de forma transparente para el usuario.

Identificador: UG-005

Nombre: Inferencia de probabilidad de presencia

Prioridad: Alta **Fuente:** Analista

Necesidad: Alta

Claridad: Alta **Verificabilidad:** Alta

Estabilidad: Media

Descripción: El núcleo del sistema analizará los tiempos de respuesta obtenidos en el UG-004. Mediante modelos estadísticos integrados, determinará a qué clase pertenece el término buscado y calculará la probabilidad matemática de que dicha información exista previamente en la base de datos de la empresa.

Identificador: UG-006

Nombre: Visualización de resultados y métricas

Prioridad: Alta **Fuente:** Cliente

Necesidad: Alta

Claridad: Alta **Verificabilidad:** Alta

Estabilidad: Alta

Descripción: El sistema presentará al usuario los resultados de la auditoría de forma gráfica. Se mostrará el porcentaje de probabilidad calculado, acompañado de gráficas que justifiquen visualmente la conclusión del análisis.

Identificador: UG-007

Nombre: Exportación de reporte de auditoría

Prioridad: Media **Fuente:** Cliente

Necesidad: Media

Claridad: Alta **Verificabilidad:** Alta

Estabilidad: Alta

Descripción: El investigador podrá descargar los resultados estadísticos obtenidos en un archivo de formato CSV. Este reporte incluirá el término consultado, la probabilidad estimada, el número de iteraciones realizadas y los tiempos medios registrados.

3.1.2. REQUISITOS NO FUNCIONALES

Identificador: UG-008

Nombre: Usabilidad e Interfaz Intuitiva

Prioridad: Alta **Fuente:** Cliente

Necesidad: Alta

Claridad: Alta **Verificabilidad:** Media

Estabilidad: Alta

Descripción: La interfaz de usuario debe estar diseñada para perfiles de auditores o investigadores que no necesariamente tienen conocimientos profundos de programación (sin necesidad de usar consola de comandos), guiando el proceso de auditoría mediante controles claros.

Identificador: UG-009

Nombre: Aislamiento y privacidad

Prioridad: Alta **Fuente:** Analistas

Necesidad: Alta

Claridad: Alta **Verificabilidad:** Media

Estabilidad: Alta

Descripción: Toda la lógica de medición de tiempos y cálculos de probabilidad debe realizarse en la máquina o servidor local donde reside el producto, sin enviar las consultas de los usuarios ni los metadatos de las empresas a servidores de terceros.

Identificador: UG-010

Nombre: Precisión temporal sin interferencias

Prioridad: Alta **Fuente:** Analistas

Necesidad: Alta

Claridad: Media **Verificabilidad:** Media

Estabilidad: Media

Descripción: El software debe estar optimizado para que los procesos de la interfaz gráfica no interfieran con el medidor de latencias interno. El cálculo de los tiempos debe ser exacto para no adulterar el modelo estadístico.

3.2. MODELO DE CASOS DE USO

Para definir el alcance completo de nuestra herramienta es muy importante la utilización del Modelo de Casos de Uso ya que nos ayuda a establecer una base sólida para la posterior implementación y validación de la plataforma. Para esto se ha llevado a cabo un proceso de análisis sobre el modelo para poder asegurar su coherencia y completa alineación con los requisitos funcionales explicados en el apartado anterior.

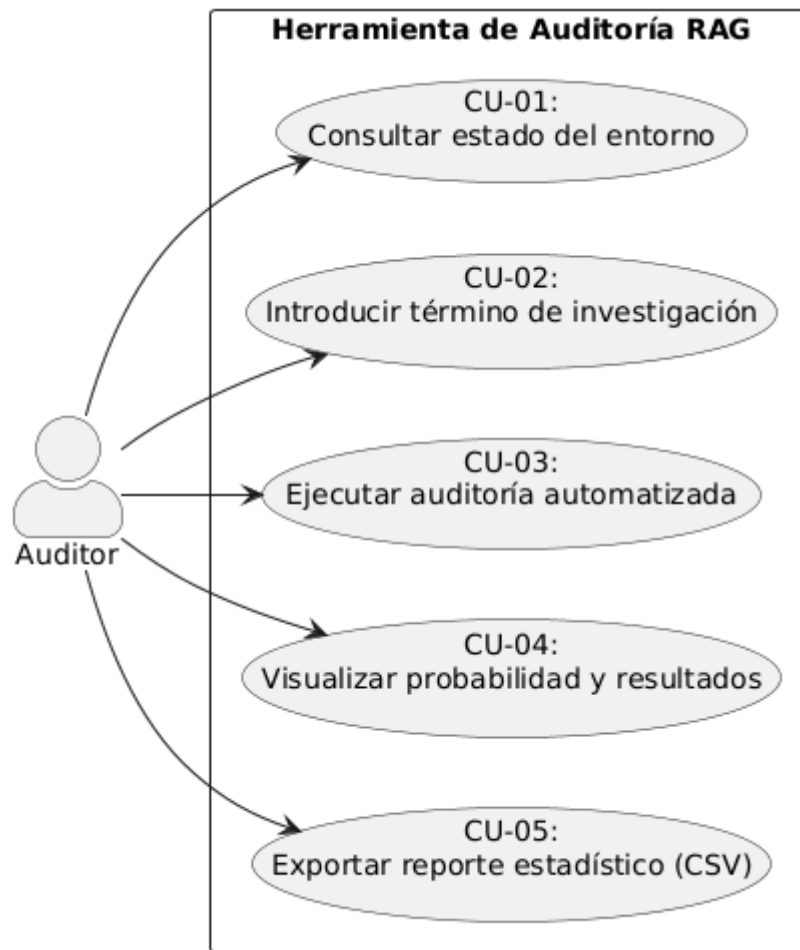
3.2.1. ACTORES

Para el desarrollo de los casos de uso lo primero es identificar que actores interactúan con el producto. En este caso el único actor involucrado será:

- **Auditor:** Representa al perfil técnico o auditor de ciberseguridad que utiliza la herramienta para evaluar si cierta información confidencial o términos específicos residen en la base de datos de una empresa, utilizando para ello el ataque por canal lateral de latencias.

3.2.2. DIAGRAMA DE CASOS DE USO

Para mostrar con mayor claridad las interacciones del actor con el sistema utilizamos el diagrama de casos de uso que podemos observar a continuación.



3.2.3. DESCRIPCIÓN DE ALTO NIVEL DE LOS CASOS DE USO

A continuación, se presenta la especificación de alto nivel para cada uno de los casos de uso identificados en el diagrama del sistema. Esta descripción permite comprender de manera rápida la interacción principal entre el actor y la plataforma, así como el propósito fundamental de cada acción.

Caso de Uso	Actor	Tipo	Descripción
CU-01	Auditor	Primario / Esencial	El usuario accede al sistema y verifica mediante indicadores visuales que la conexión con la base de datos vectorial está operativa antes de iniciar cualquier proceso de auditoría.
CU-02	Auditor	Primario / Esencial	El usuario interactúa con la interfaz gráfica para introducir la palabra clave, término o concepto específico cuya probabilidad de existencia desea auditar en el entorno objetivo.

CU-03	Auditor	Primario / Esencial	El usuario acciona el inicio de la prueba. El sistema automatiza las iteraciones de consulta necesarias, explotando el canal lateral de latencias en segundo plano para obtener una muestra de tiempos estadísticamente relevante.
CU-04	Auditor	Primario / Esencial	Una vez finalizada la prueba, el sistema presenta al usuario el porcentaje de probabilidad de presencia del término buscado, apoyándose en representaciones gráficas para facilitar su interpretación.
CU-05	Auditor	Secundario / Opcional	El usuario solicita descargar los resultados de la sesión. El sistema genera un archivo en formato CSV que contiene los datos en bruto de los tiempos y el veredicto, permitiendo su uso en informes de auditoría externos.

3.3. MATRIZ DE TRAZABILIDAD

A partir del modelo de casos de uso, se ha construido la Matriz de Trazabilidad que garantiza la cobertura total de todos los requisitos del sistema. Esta matriz muestra de manera visual cómo todos los requisitos definidos son cubiertos por al menos un caso de uso del modelo final. Esto garantiza que el alcance del proyecto está completamente definido y validado. Una «X» en la intersección indica que el caso de uso de la columna satisface, implementa o se ve afectado por el requisito de la fila correspondiente.

Requisitos / Casos de Uso	CU-01	CU-02	CU-03	CU-04	CU-05
UG-001 Introducción de términos de consulta		X			
UG-002 Visualización del estado del sistema	X				
UG-003 Cálculo automático de iteraciones necesarias			X		
UG-004 Ejecución automatizada de la auditoría			X		

UG-005 Inferencia de probabilidad de presencia			X	X	
UG-006 Visualización de resultados y métricas				X	
UG-007 Exportación de reporte de auditoría					X
UG-008 Usabilidad e Interfaz Intuitiva	X	X	X	X	X
UG-009 Aislamiento y privacidad	X	X	X	X	X
UG-010 Precisión temporal sin interferencias			X		